

UART Serial Communication Controller Using FPGA

Course Code -BECE102L

Guide Name - Dr. PALLA PENCHALIAH

Presented By:

Nicole T - 24BEC0387

Akhila N - 24BML0078

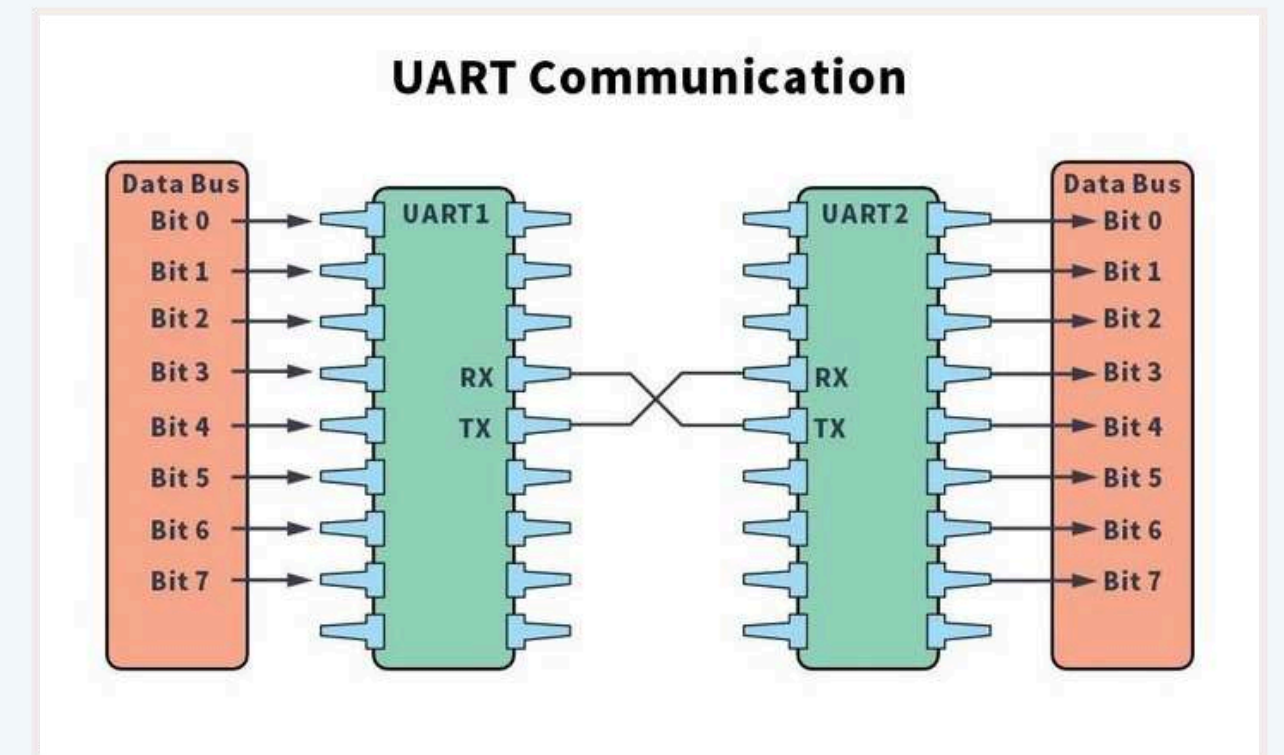
Dishika - 24BEC0486

Gurupriyaa - 24BEC0139



Contents

- 03. INTRODUCTION
- 04. AIM & OBJECTIVES
- 05. BLOCK DIAGRAM & FLOW CHART
- 08. WORKING & SYSTEM DESIGN
- 11. IMPEMANETATION USING VERILOG HDL
- 21. ADVANTAGES



Introduction

What is UART?

- UART stands for Universal Asynchronous Receiver and Transmitter.
- It is a hardware communication protocol that allows two devices to send/receive data serially (bit by bit) without needing a shared clock.
- The main purpose of the UART (Universal Asynchronous Receiver/Transmitter) protocol is to convert parallel data into serial data for transmission and convert serial data back into parallel data upon reception
 - The UART's job is to convert 8-bit parallel data into a 10-bit serial stream
 - Normally UART is built into microcontrollers, but in FPGA we design it ourselves using Verilog.

Aim

To design and implement a UART serial communication controller on an FPGA platform using Verilog

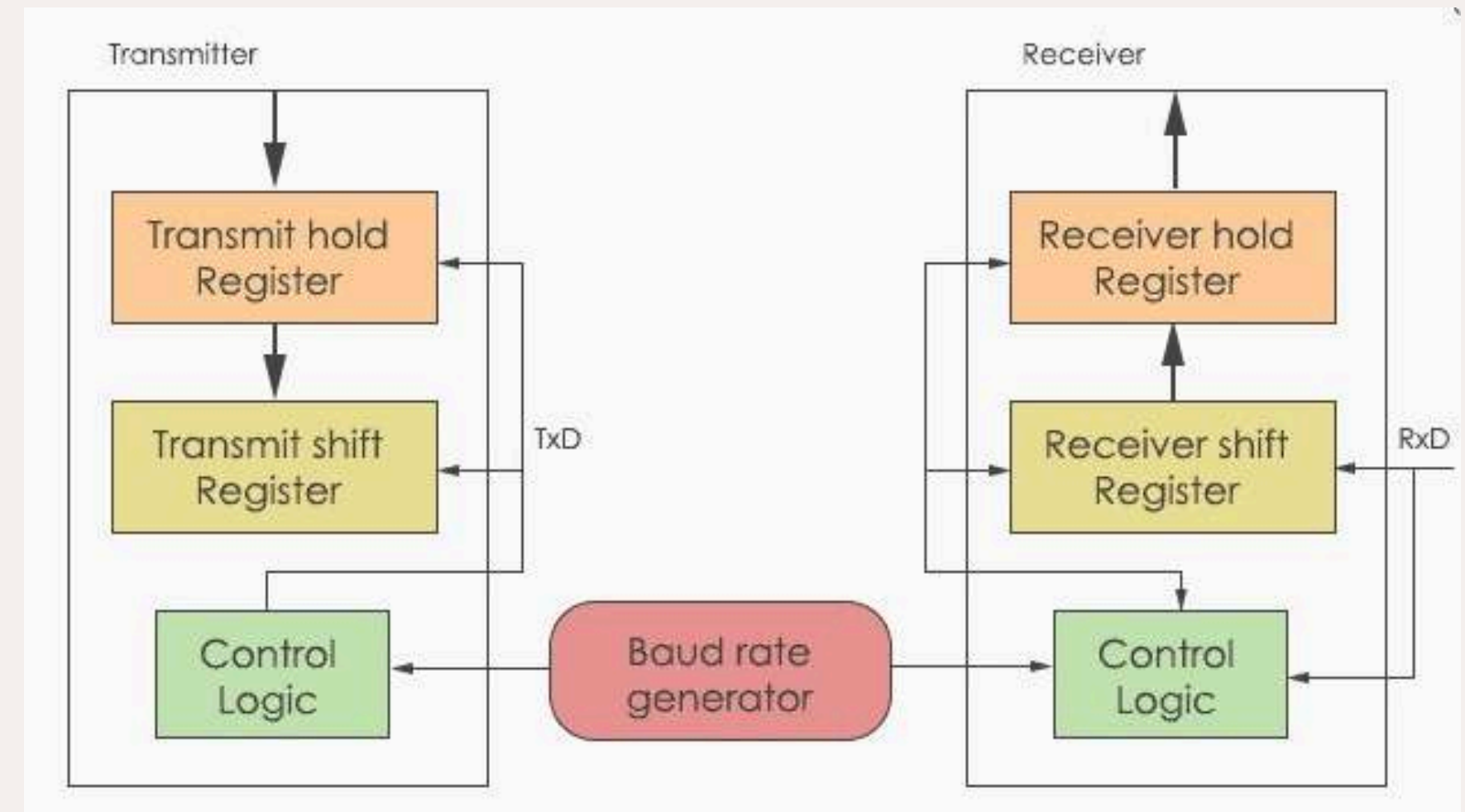
HDL, simulate its functionality using ModelSim/Vivado, and demonstrate reliable serial data transmission and reception between digital systems.

Objectives

- Understand how serial communication works at the hardware level.
- Learn to model and simulate real-time digital communication systems.
- Apply Verilog HDL for hardware implementation of UART.
- Verify design functionality using simulation tools.

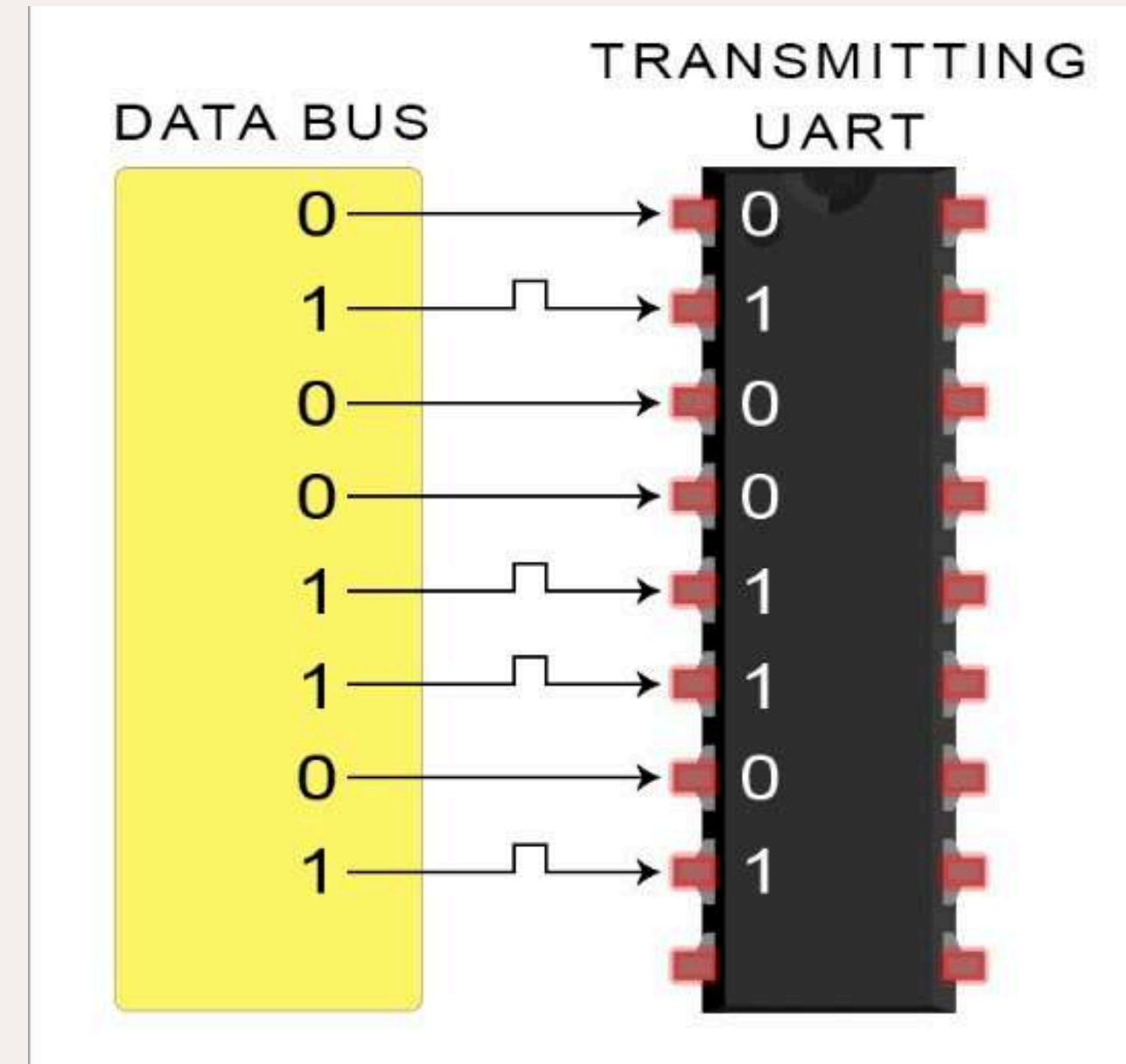
Block Diagram:

UART Transmitter(TX)
UART Receiver (RX)
Baud Rate Generator
TopModule / ControlUnit
Output Display



Block Diagram:

UART Transmitter(TX)
UART Receiver (RX)
Baud Rate Generator
TopModule / ControlUnit
Output Display

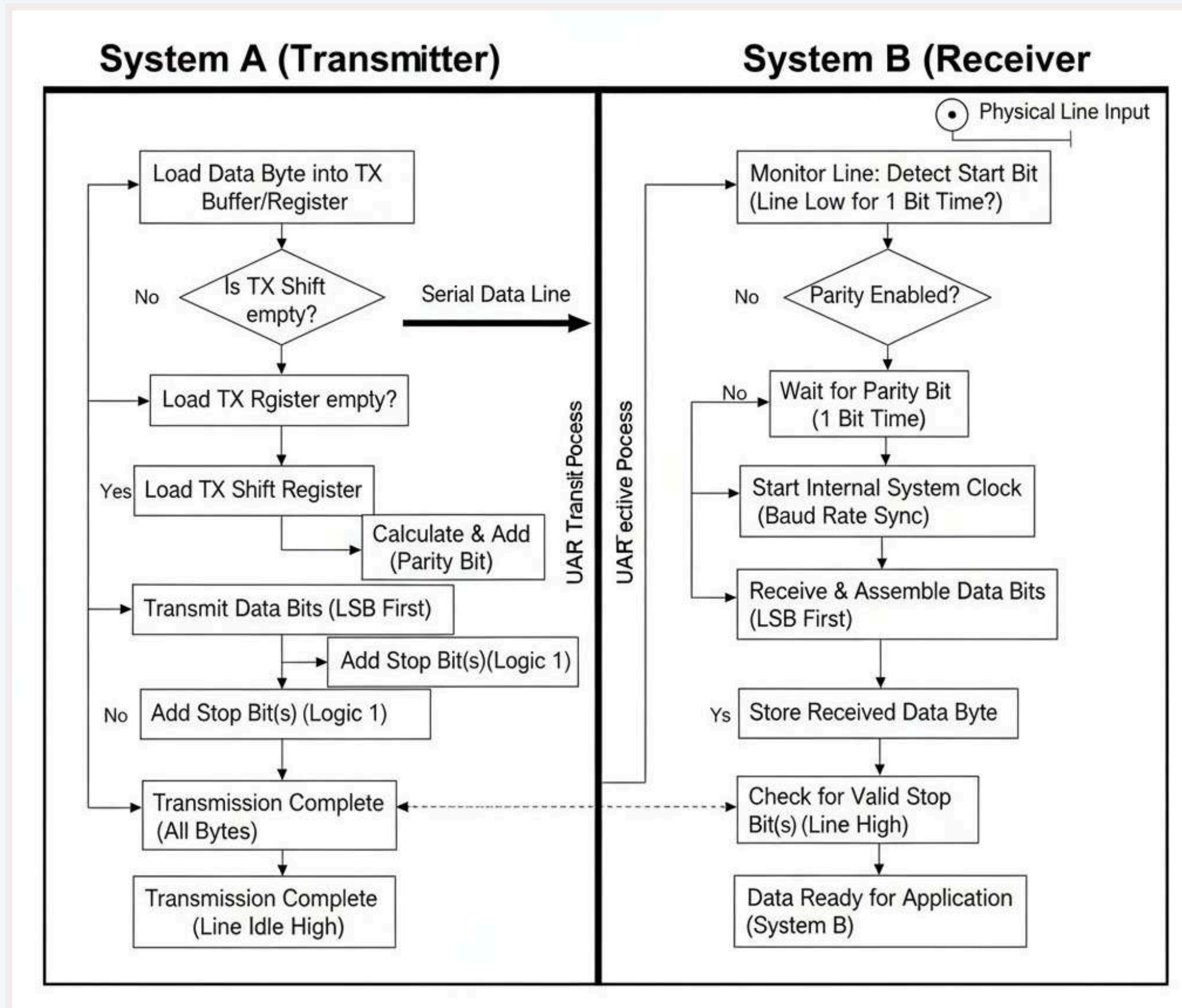


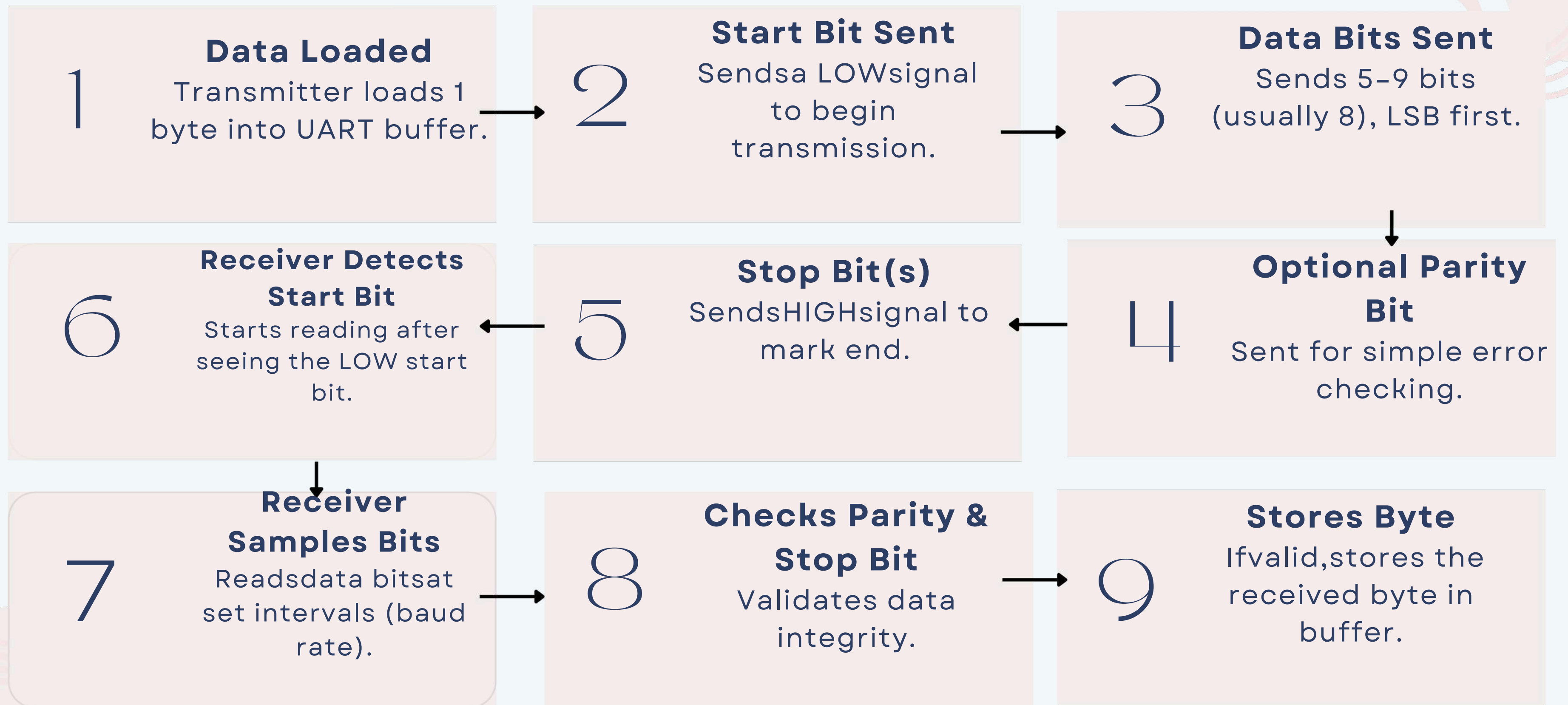
Truth Table

- The truth table for UART defines how input signals like clock, reset, baud rate tick, and transmit start affect output signals such as serial data output, transmitter busy status, received data, and receive valid flag, ensuring proper asynchronous serial communication.
- It is essential for verifying UART operation, debugging transmitter and receiver behavior, and ensuring reliable data transmission and reception by showing expected signal relationships and states during communication

clk	rst	baudtick	txstart	serialout	rxdata	rxvalid
1	0	0	0	0	1	0
2	1	0	0	1	1	0
baudtick	0	0	1	1	1	0
	0	1	1	1	0	0
	1	0	1	0	0	0
	1	0	1	0	0	0
UAAT	1	0	1	0	1	1
	1	0	0	0	0	1
	1	0	0	0	1	1

FlowChart





Working

The UART protocol is a foundational method for device communication that efficiently converts fast, multi-wire parallel data (like 8 bits at once) into a slow, single-wire serial data stream. The working principle is designed for reliability without a shared clock: both the sender (TX) and receiver (RX) modules must simply be set to the same speed (Baud Rate). The TX module begins every transmission by sending a low Start Bit, which acts as the explicit trigger for the RX's timing counter. Following this, the RX's crucial logic is to deliberately delay its sample until the center of the bit-time. This center-sampling method is what ensures data integrity by mitigating timing errors (clock drift) between the two independent devices, accurately capturing the 8 data bits and the final high Stop Bit before successfully reconstructing the full parallel byte.

System design

UART Transmitter (TX):

- Accepts parallel data input from the FPGA (for example, 8-bit data).
- Converts the parallel data into a serial bitstream.
- Adds the start bit (logic 0), data bits (LSB first), optional parity bit (for error checking), and stop bit (logic 1) to frame the data correctly.
- Transmits data over the TX line at the specified baud rate (e.g., 9600 bps).

UART Receiver (RX):

- Monitors the incoming RX line for the start bit.
- Samples the incoming serial data at the defined baud rate.
- Reconstructs the original parallel data word from the received serial stream.
- Verifies the stop bit and optional parity to ensure correct reception.
- Provides the received data as parallel output to the FPGA system.

Baud Rate Generator:

- Divides the high-frequency FPGA clock (e.g., 50 MHz or 100 MHz) into a lower clock matching the required baud rate (e.g., 9600 bps).
- Provides timing signals for both transmitter and receiver.

Top Module / Control Unit:

- integrate transmitter, receiver, and baud generator.
- Provides global control signals like clock and reset.
- Handles input/output data flow between FPGA and external device.

Output Display:

- In simulation, received data can be displayed in the waveform or console.
- On FPGA hardware, received data may be sent to LEDs, seven-segment displays, or transmitted back to PC for verification.

Implementation using Verilog HDL

UART_TOP

```
module uart_top(  
    input        clk,  
    input        rst,  
    input  [7:0] tx_data,  
    input        tx_start,  
    output       tx_busy,  
    output       serial_out,  
    input        serial_in,  
    output  [7:0] rx_data,  
    output       rx_valid  
);  
    wire tx_tick;  
    wire rx_tick;  
    baud_gen u_baud_gen (  
        .clk(clk),  
        .rst(rst),  
        .tx_tick(tx_tick),  
        .rx_tick(rx_tick)  
    );  
    uart_tx u_uart_tx (  
        .clk(clk),  
        .rst(rst),  
        .baud_tick(tx_tick),  
        .tx_start(tx_start),  
        .tx_data(tx_data),  
        .serial_out(serial_out),  
        .tx_busy(tx_busy)  
    );
```

BAUD_GEN

```
module baud_gen(  
    input  clk,  
    input  rst,  
    output reg tx_tick,  
    output reg rx_tick  
);  
  
localparam TX_DIVISOR = 16'd5207;  
localparam RX_DIVISOR = 9'd325;  
  
reg [15:0] tx_counter;  
reg [8:0]  rx_counter;  
  
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        tx_counter <= 16'd0;  
        rx_counter <= 9'd0;  
        tx_tick    <= 1'b0;  
        rx_tick    <= 1'b0;  
    end else begin  
  
        if (tx_counter == TX_DIVISOR) begin  
            tx_counter <= 16'd0;  
            tx_tick    <= 1'b1;  
        end else begin  
            tx_counter <= tx_counter + 1;  
            tx_tick    <= 1'b0;  
        end  
    end  
end
```

UART_TX

```
module uart_tx(
    input        clk,
    input        rst,
    input        baud_tick,
    input        tx_start,
    input [7:0]  tx_data,
    output reg   serial_out,
    output reg   tx_busy
);
    localparam STATE_IDLE = 3'b000;
    localparam STATE_START = 3'b001;
    localparam STATE_DATA = 3'b010;
    localparam STATE_STOP = 3'b011;

    reg [2:0] state;
    reg [2:0] bit_index;
    reg [7:0] data_reg;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            state      <= STATE_IDLE;
            bit_index  <= 3'd0;
            serial_out <= 1'b1;
            tx_busy    <= 1'b0;
            data_reg   <= 8'd0;
        end else begin

            case (state)

                STATE_IDLE: begin
```


UART_RX

```
module uart_rx(  
    input        clk,  
    input        rst,  
    input        baud_tick_16x,  
    input        serial_in,  
    output reg [7:0] rx_data,  
    output reg    rx_valid  
);  
  
localparam STATE_IDLE = 3'b000;  
localparam STATE_START = 3'b001;  
localparam STATE_DATA = 3'b010;  
localparam STATE_STOP = 3'b011;  
  
reg [2:0] state;  
reg [3:0] sample_counter;  
reg [2:0] bit_index;  
reg [7:0] data_reg;  
reg rx_valid_prev;  
  
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        state          <= STATE_IDLE;  
        sample_counter <= 4'd0;  
        bit_index       <= 3'd0;  
        data_reg        <= 8'd0;  
        rx_valid        <= 1'b0;  
        rx_valid_prev   <= 1'b0;  
    end else begin  
  
        if (rx_valid_prev) begin
```

TB_UART_TOP

```
module tb_uart_top;
    reg clk;
    reg rst;
    reg [7:0] tx_data;
    reg tx_start;
    wire tx_busy;
    wire serial_out;
    wire [7:0] rx_data;
    wire rx_valid;
    wire serial_loopback = serial_out;
    uart_top uut (
        .clk(clk),
        .rst(rst),
        .tx_data(tx_data),
        .tx_start(tx_start),
        .tx_busy(tx_busy),
        .serial_out(serial_out),
        .serial_in(serial_loopback),
        .rx_data(rx_data),
        .rx_valid(rx_valid)
    );
    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end
    initial begin
        $display("Starting Testbench...");
        rst = 1;
        tx_start = 0;
        tx_data = 8'h00;
        #100;
    end
endmodule
```

Compilation

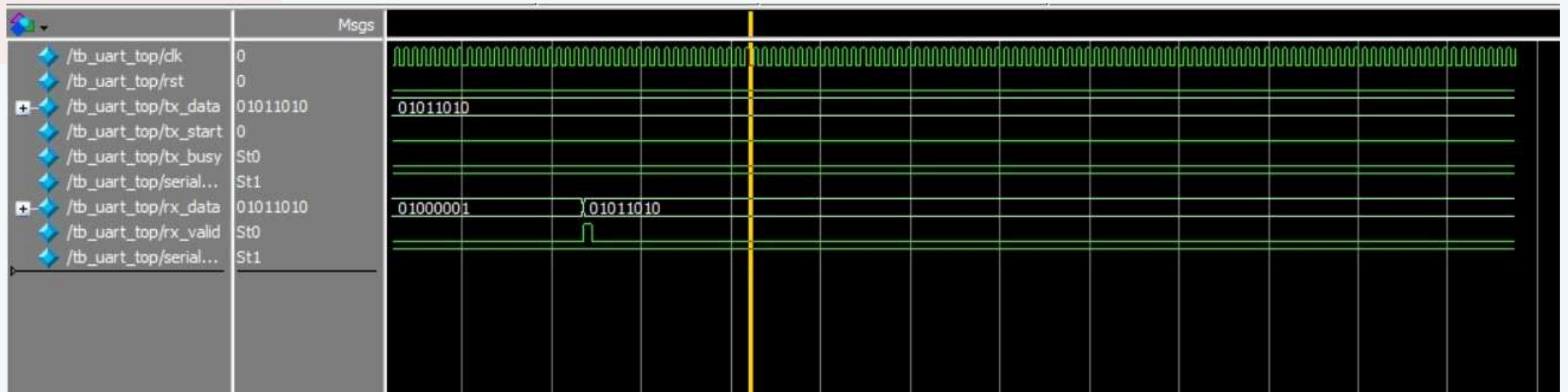
Name	Status	Type	Order	Modified
 baud_gen.v	✓	Verilog	3	10/23/2025 10:38:41 ...
 uart_rx.v	✓	Verilog	1	10/24/2025 11:13:11 ...
 uart_top.v	✓	Verilog	4	10/24/2025 11:15:31 ...
 uart_tx.v	✓	Verilog	2	10/24/2025 11:32:41 ...
 tb_uart_top.v	✓	Verilog	0	10/24/2025 11:42:12 ...

Run time transcript:

```
VSIM 2> run -all
# Starting Testbench...
# Sending 'A' (0x41)...
# TX busy...
# TX idle.
# RX valid!
# SUCCESS: Received 'A' (0x41)
# Sending 'Z' (0x5A)...
# SUCCESS: Received 'Z' (0x5A)
# Testbench finished.
# ** Note: $stop      : C:/Users/HP/Downloads/uart project/tb_uart_top.v(75)
#    Time: 2397310 ns  Iteration: 0  Instance: /tb_uart_top
# Break in Module tb_uart_top at C:/Users/HP/Downloads/uart project/tb_uart_top.v line 75
```

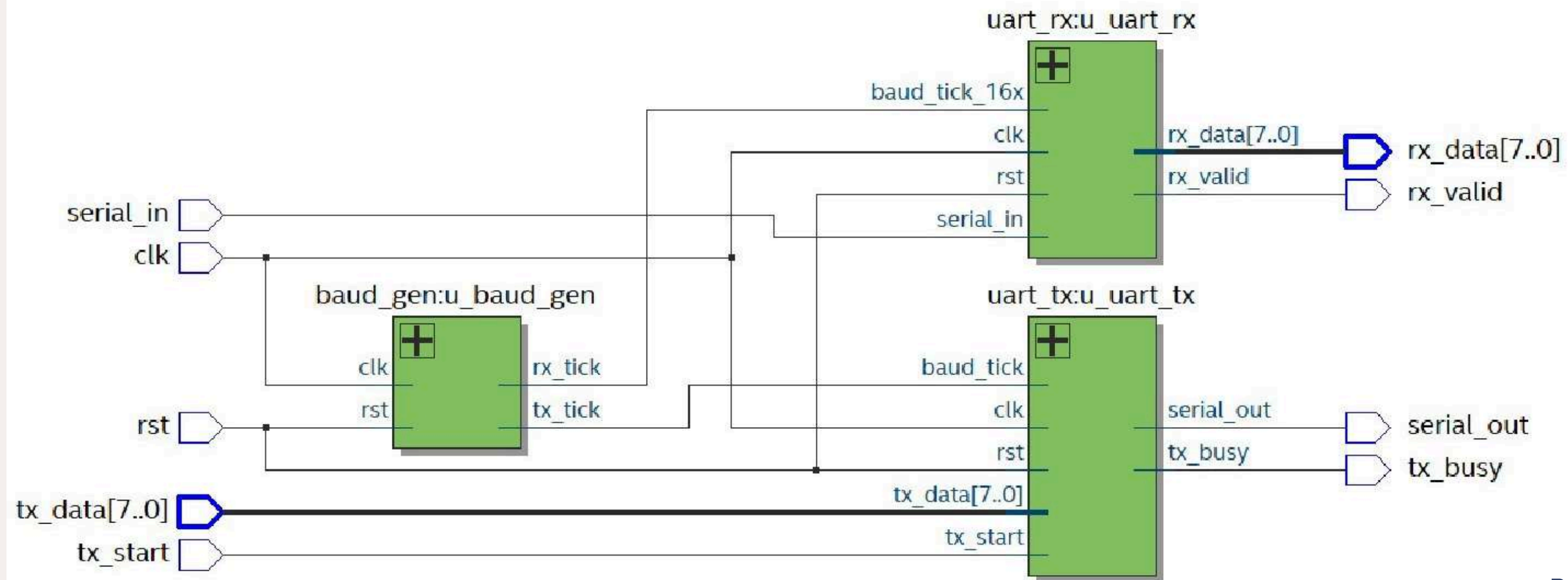
```
73
74      $display("Testbench finished.");
75      $stop;
76  end
77
78 endmodule
```


Simulation:



Schematic

Analysis & Synthesis: (Quartus)



Advantages

- Reusability: A UART module's HDL code can be applied to various projects and FPGA devices, which saves time when creating new designs.
- Customization and Flexibility: By altering the HDL code and re-synthesizing the design, you can quickly change the UART protocol's baud rate, data bits, parity, and stop bits. In contrast to a fixed-function UART chip, this enables you to customize the communication to a particular application.
- Integration: A larger FPGA-based system can directly incorporate the UART functionality. This reduces board space, simplifies PCB design, and lowers the bill of materials by doing away with the need for an external UART chip.
- Debugging and Prototyping: FPGAs are great for debugging and prototyping.

Thank You



A PROJECT BY :

Dishika - 24BECO486
Gurupriyaa - 24BECO139

Akhila N - 24BML0078
Nicole T - 24BECO837

THANKS!

PAGE 22

UART Serial Communication Controller Using FPGA

G.Dishika

N.Gurupriyaa

Dr.Palla Penchalaih

Vellore Insstitute of Technology

Vellore,Tamil Nadu

navaneethakrishnan.2024@vitst

udent.ac.in

Akhila N

NicoleT

Dr.Palla Penchalaih

Vellore Institute of Technology

Vellore,Tamil Nadu

akhila.n2024@vitstudent.ac.in

Abstract :

This project focuses on designing and implementing a Universal Asynchronous Receiver and Transmitter (UART) using digital logic on an FPGA platform. The main goal is to establish reliable serial communication between the FPGA and external devices such as microcontrollers or computers. The proposed system consists of a UART transmitter and receiver module that operates at a fixed baud rate and follows the standard communication format with start, data, parity, and stop bits. The design is written in Verilog HDL, simulated using ModelSim, and then synthesized and tested on an FPGA board. The developed UART communicator achieves efficient data transmission and reception with minimal delay and precise timing synchronization. Overall, this project demonstrates the flexibility, speed, and hardware-level control that FPGA-based communication systems offer compared to conventional microcontroller-based UART implementations.

Keywords— FPGA, UART, Serial Communication, Verilog HDL, Digital Logic, Data Transmission.

I. INTRODUCTION

Serial communication plays a vital role in digital systems for transferring data between devices using fewer signal lines. Among various serial communication protocols, UART is one of the simplest and most commonly used. It enables asynchronous data exchange without requiring a shared clock line, relying instead on synchronized timing between the transmitter and receiver. In conventional systems, UARTs are implemented using microcontrollers or dedicated ICs; however, FPGA-based implementations offer improved timing control, customization, and parallel processing capabilities. This project aims to design a UART serial communicator using Verilog HDL to achieve accurate and efficient transmission of digital data between FPGA and other serial devices. The implementation involves developing transmitter and receiver logic modules, integrating them with a baud rate generator, and validating performance through simulation and hardware testing. The resulting design demonstrates how UART communication can be realized entirely through digital logic within an FPGA..

II. PROBLEM STATEMENT

In most embedded and digital systems, data communication between devices such as microcontrollers, sensors, or computers is essential. Conventional UART modules are usually built into microcontrollers, which limits flexibility and hardware-level customization. Additionally, software-based serial communication often suffers from timing errors, latency issues, and limited control over bit-level operations. To overcome these challenges, this project proposes a **hardware-based UART communicator implemented on an FPGA** using Verilog HDL. The FPGA's reconfigurable logic allows precise control of timing, baud rate generation, and data framing, ensuring faster and more reliable serial communication. The design provides a better understanding of how UART works internally while offering improved performance compared to microcontroller-based approaches.

III. OBJECTIVES

The main objectives of this project are:

- To design and implement a UART serial communication system on an FPGA using Verilog HDL.
- To establish reliable data transmission and reception between the FPGA and external serial devices.
- To develop transmitter and receiver modules that handle start, data, parity, and stop bits according to standard UART protocol.

- To simulate and verify the UART operation using ModelSim before hardware implementation.
- To synthesize and test the design on an FPGA development board, ensuring accurate baud rate and low-latency communication.

IV. METHODOLOGY

In the proposed system, the Universal Asynchronous Receiver and Transmitter (UART) is implemented using digital logic on an FPGA board. The complete methodology involves the design, simulation, and hardware implementation of UART transmitter and receiver modules for serial data communication.

UART Overview:

UART is a serial communication protocol that transmits data bit by bit over a single wire. It uses start, data, parity, and stop bits to ensure reliable data transfer between two devices. The communication is asynchronous, meaning it does not require a separate clock line — both the transmitter and receiver operate at a common baud rate.

TransmitterSection:

The UART transmitter converts 8-bit parallel data into a serial data stream. The transmitter adds a start bit (logic 0) before the data bits and a stop bit (logic 1) after the data frame. Optionally, a parity bit can be added for error checking. Each bit is transmitted at a fixed interval determined by the baud rate generator.

ReceiverSection:

The receiver monitors the serial input line for a start bit. Once detected, it samples the incoming bits at specific intervals to reconstruct the 8-bit data frame. After reading all bits, the receiver checks the parity (if enabled) and stop bits to validate the received data. The final 8-bit data is then provided as parallel output.

BaudRateGenerator:

A baud rate generator module is designed using a frequency divider circuit to produce timing pulses corresponding to the selected baud rate (for example, 9600 bps). These pulses control the timing for both transmission and reception, ensuring synchronization between sender and receiver.

- **Simulation and Verification:** The transmitter, receiver, and baud rate generator modules are coded in Verilog HDL. The complete UART system is simulated using ModelSim to verify correct signal timing, bit synchronization, and data accuracy. The waveforms of Tx and Rx signals are analyzed to confirm proper communication between the modules.

- **Hardware Implementation:** After successful simulation, the UART design is synthesized and programmed onto the FPGA development board (such as DE2-115). The FPGA is connected to a computer through a USB-to-serial interface (RS-232 or FTDI converter). Serial communication is tested using terminal software to verify data transmission and reception in real-time.

- **Testing and Observation:** The system is tested by transmitting characters from the FPGA to the PC terminal and vice versa. The results are observed on the terminal window to ensure correct character display and timing accuracy. Any communication errors are analyzed by checking waveform signals on the ModelSim simulation output and FPGA debug LEDs.

Edge Detection Logic: Edge Detection Logic: Edge-triggered flip-flops and sequential logic are used by the Verilog HDL implementation of the UART controller to precisely synchronize data transmission and reception. By limiting data updates to certain clock signal transitions, edge-triggering

reduces timing risks and guarantees consistent responses from storage components. In order to counteract clock skew, the UART system maintains consistent bit durations and ideal sampling times by having both transmitter and receiver modules run on the positive edge of the clock or baud tick. Strong and dependable UART communication that works with a variety of FPGA platforms is ensured by the baud rate generator, which counts clock edges to specify timing intervals.

OPERATIONAL FLOW

The primary operational flow of the Universal Asynchronous Receiver and Transmitter (UART) controller implemented in Verilog HDL consists of initialization, data transmission, and reception processes. The four primary modules built into the design are the baud rate generator, UART transmitter (TX), UART receiver (RX), and a top-level control module that oversees their operation. The simulation was conducted using ModelSim/Quartus to verify that the Verilog design was operating as intended.

The top module activates the system clock and asynchronous reset signals to ensure that all internal registers and buffers are cleared during initialization. A lower-frequency baud signal, typically 9600 bps, is then produced by dividing the high-frequency FPGA clock (for example, 50 MHz) by the baud rate generator. By specifying the timing intervals for data transmission and reception, this baud signal guarantees synchronized serial communication in the absence of a shared external clock.

The UART transmitter transforms the parallel 8-bit data into a framed serial bitstream when a data byte is prepared for transmission. A start bit (logic 0), eight data bits (LSB first), an optional parity bit, and one or more stop bits (logic 1) make up each frame. The transmitter transforms parallel data from the FPGA into a serial data stream on the TX line by sequentially outputting each bit at each baud tick.

The UART receiver continuously looks for a falling edge on the RX line to determine the start bit. After detection, the receiver samples incoming bits at the center of each baud interval to reconstruct the original data word. The received frame is then checked for parity and stop-bit validity to ensure data integrity. If no framing or parity errors are detected, the reconstructed 8-bit parallel data is stored in the receiver buffer and transmitted to the top module for further processing.

The functionality of the Verilog design is confirmed by waveform analysis during simulation, which confirms that the data sent and received are identical. This operational flow demonstrates the reliable implementation of asynchronous serial communication on FPGA hardware using Verilog HDL.

uart communication protocol have been illustrated in the Fig:1.

VI. WORKING PRINCIPLE (Based on Flowchart)

Digital systems can reliably exchange data without a shared clock signal thanks to the Universal Asynchronous Receiver and Transmitter (UART), a basic serial communication protocol. The asynchronous transfer of data bits framed with particular start and stop conditions is the foundation of the UART controller's operation. The receiver carries out the opposite operation to reconstruct the original data after the transmitter transforms parallel binary data into a serial bitstream.

In this design, which is implemented using Verilog HDL on an FPGA platform, the UART operates through a transmitter (TX), receiver (RX), and a baud rate generator that ensures synchronized timing. The transmitter instructs the receiver to begin sampling at the beginning of each transmission by asserting a start bit (logic 0). Following the start bit, eight data bits are sent in a sequential order, beginning with the least significant bit (LSB). Depending on the setup, a parity bit could be added to offer simple error detection. The transmission frame's end is indicated by one or more stop bits (logic 1).

The receiver continuously checks the serial input line for the high-to-low transition of the start bit. It begins a timing counter based on the generated baud rate and samples each subsequent bit at the center of its period after detection to minimize timing errors caused by clock mismatches. After receiving the complete frame and confirming the stop bit and optional parity, the receiver transforms the serial data back into its 8-bit parallel form. The reconstructed data can then be processed or shown by the system.

This concept illustrates how UART uses the least amount of hardware to enable dependable serial communication between two asynchronous devices. It is possible to modify and validate the internal timing, framing, and control logic of the UART through simulation by putting the design on an FPGA using Verilog HDL. This ensures precise data transfer between digital systems

I. FLOWCHART

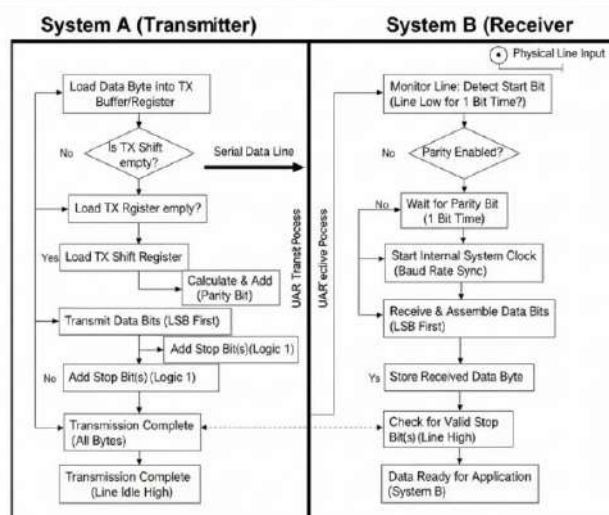


Fig:1.Flowchart of the setup.

The working of the components in the FPGA-BASED for

TOOLS USED:

Component Description / Function

FPGA Board	Intel/Altera MAX 10 FPGA used as the target hardware for implementation.
Quartus Prime Software	Used for synthesis, pin assignment, and generation of the bitstream file.
ModelSim (Intel Edition)	Used to simulate the UART design and verify data transmission and reception waveforms.
Verilog HDL	Hardware description language used to model the UART's functional modules (TX, RX, baud rate generator, and control unit).
Clock Source	50 MHz on-board oscillator divided by the baud generator to achieve a baud rate of 9600 bps.
Peripheral Devices	Serial data interface (TX/RX lines), optional LEDs or 7-segment display for output verification .

Test setup

An FPGA development board was used in the test setup, and it was serially interfaced to a computer. Quartus Prime was used to integrate and compile the Verilog modules `uart_tx`, `uart_rx`, `baud_gen`, and `uart_top`. A behavioral testbench (`tb_uart_top.v`) was used in ModelSim to perform functional verification.

An 8-bit data word was applied to the transmitter input during the simulation. Timing pulses were supplied by the baud rate generator to synchronize the RX and TX modules. To verify end-to-end serial communication, the transmitter output (`tx_line`) was looped back to the receiver input (`rx_line`). The start bit, eight data bits, and stop bit transitions corresponding to the selected baud rate were shown in the simulation waveforms. To verify accuracy, the transmitted data and the reconstructed parallel data at the receiver output were compared.

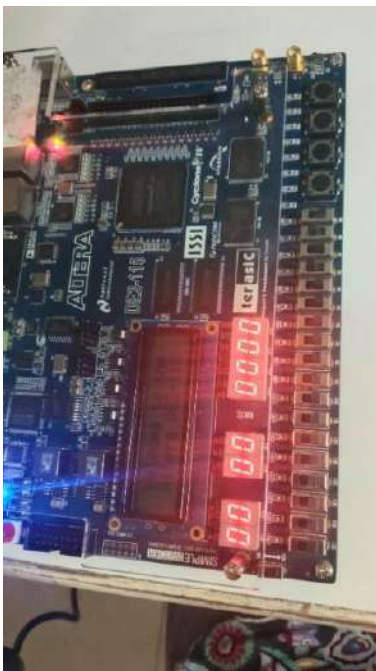
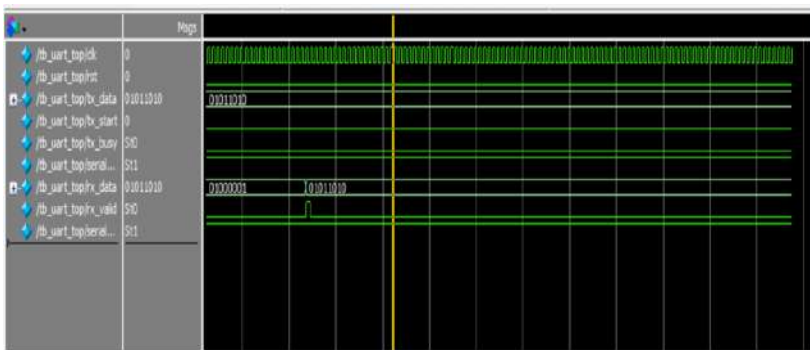
RESULT

The accuracy of the UART design's asynchronous serial communication between the transmitter and receiver modules was confirmed by simulation results. One start bit (logic 0), eight data bits (LSB first), and one stop bit (logic 1) made up the transmitter waveform, which showed the proper frame structure. The precise bit sampling in the receiver waveform matched the baud tick produced by the baud rate divider.

The transmitted and received data matched exactly during the loopback test, demonstrating proper reconstruction, synchronization, and bit framing. The FPGA's 50 MHz reference was successfully converted into a 9600 bps clock by the baud rate generator. The sampling logic successfully recorded data at the mid-bit interval, as evidenced by the absence of frame or parity errors.

The efficiency of the Verilog implementation is confirmed by these simulation results. The design can be expanded to incorporate features like error detection, multi-byte buffering, or parity generation for more intricate communication systems, and it can be synthesized on FPGA hardware with little time constraints.

```
VSIM2> run -all
# Starting Testbench...
# Sending 'A' (0x41)...
# TX busy...
# TX idle...
# RX valid!
# SUCCESS: Received 'A' (0x41)
# Sending 'Z' (0x5A)...
# SUCCESS: Received 'Z' (0x5A)
# Testbench finished.
** Note: $stop : C:/Users/NP/Downloads/uart project/tb_uart_top.v(75)
# Time: 2397310 ns Iteration: 0 Instance: /tb_uart_top
# Break in Module tb_uart_top at C:/Users/NP/Downloads/uart project/tb_uart_top.v line 75
```



VII. ADVANTAGES OF OUR SYSTEM

This FPGA-based step counter implementation presents several clear advantages over traditional microcontroller-based pedometers.

- **Reusability:** A UART module's HDL code can be applied to various projects and FPGA devices, which saves time when creating new designs.

- Customization and Flexibility: By altering the HDL code and re-synthesizing the design, you can quickly change the UART protocol's baud rate, data bits, parity, and stop bits. In contrast to a fixed-function UART chip, this enables you to customize the communication to a particular application.
- Integration: A larger FPGA-based system can directly incorporate the UART functionality. This reduces board space, simplifies PCB design, and lowers the bill of materials by doing away with the need for an external UART chip.
- Debugging and Prototyping: FPGAs are great for debugging and prototyping.

VIII. CONCLUSION

The design of a dependable serial communication system on an FPGA platform is successfully demonstrated by the implementation of the Universal Asynchronous Receiver and Transmitter (UART) controller using Verilog HDL. Without the need for a shared clock, the developed system effectively transforms parallel data into serial form for transmission and back into parallel form upon reception. The design's ability to transfer data accurately at the specified baud rate of 9600 bps was confirmed by means of methodical ModelSim simulation and Quartus Prime synthesis.

The UART modules—transmitter, receiver, and baud rate generator—functioned cohesively to ensure proper bit framing, synchronization, and error-free communication. The simulation results confirmed that the data bit timing, start and stop bit detection, and parity verification were all done correctly. In larger digital or embedded systems, this demonstrates that the Verilog-based UART controller can function as a basic communication block.

Future scope

Higher performance requirements and more sophisticated communication features can be supported by expanding the suggested UART design. Potential improvements consist of:

Improvements to Parity and Error Detection: Including framing and overrun error flags for reliable communication, as well as parity generation and checking.

Integrating transmit and receive FIFOs to allow for continuous data transfer without processor intervention is known as FIFO buffer integration.

Configurable Frame Format and Baud Rate: Using programmable control registers, baud rate, data length, parity, and stop bits can be dynamically changed while the system is operating.

Enabling interrupt signaling for transmit/receive completion in real-time embedded applications is known as interrupt-driven operation.

Hardware Implementation and Testing: Installing the synthesized design on an actual FPGA board and connecting it to external devices, like PCs or microcontrollers, using USB-to-UART or RS-232 converters.

REFERENCES

1. [1] P. Sharma, S. Rani, and A. Kumar, "Design and Implementation of UART Using Verilog HDL," *International Journal of Advanced Research*, vol. 10, no. 7, pp. 98–104, 2025. [Online]. Available: https://www.journalijar.com/uploads/2025/07/6892e0ba8fc6e_IJA R-53012.pdf
2. [2] M. A. Rahman and H. Li, "Design and Implementation of UART Based on Verilog HDL," *ResearchGate*, 2023. [Online].

Available:

https://www.researchgate.net/publication/369465619_Design_And_Implementation_of_UART_Based_on_Verilog_HDL

3. [3] R. Gupta, "Digital System Design with FPGA: Implementation Using Verilog HDL," *ProjectFPGA Technical Report*, 2022. [Online]. Available:

<https://projectfpga.com/resources/fpga-implementation.pdf>

4. [4] M. S. Kiran and B. Rao, "Design of UART Protocol Using Verilog HDL," *International Journal of Computational Engineering and Science (IJCESN)*, vol. 9, no. 3, pp. 51–58, 2024. [Online]. Available:

<https://ijcesn.com/index.php/ijcesn/article/download/3306/1037/9110>

5. [5] T. Ramesh and S. Narayanan, "UART Implementation with BIST Using Verilog HDL," *International Journal of Computer Science and Mobile Computing (IJCSMC)*, vol. 5, no. 12, pp. 40–48, Dec. 2016. [Online]. Available:

<https://ijcsmc.com/docs/papers/December2016/V5I12201605.pdf>

6. [6] "Help: UART Protocol – Complete but Basic Explanation," *Digilent Forum Discussion*, 2024. [Online]. Available:

<https://forum.digilent.com/topic/24547-help-uart-protocol-complete-but-basic-explanation-and-guides-from-newbie-perspective/>

7. [7] Chipmunk Logic, "Design of UART Controller in Verilog/VHDL," *Chipmunk Logic Blog*, 2023. [Online]. Available:

<https://chipmunklogic.com/ip-cores/design-of-uart-controller/>

8. [8] Wikipedia, "Flip-Flop (Electronics)," *Wikipedia: The Free Encyclopedia*, 2025. [Online]. Available:

[https://en.wikipedia.org/wiki/Flip-flop_\(electronics\)](https://en.wikipedia.org/wiki/Flip-flop_(electronics))

9. [9] EngineersGarage, "How to Design a T Flip-Flop in Verilog," *EngineersGarage Tutorials*, 2022. [Online]. Available:

<https://www.engineersgarage.com/how-to-design-a-t-flip-flop-in-verilog/>

10. [10] M. A. D'Addio and M. T. Moreira, "Design of a Ternary Edge-Triggered D Flip-Flap-Flop for Multiple-Valued Sequential Logic," *arXiv preprint arXiv:1609.03897*, 2016. [Online]. Available:

<https://arxiv.org/abs/1609.03897>

11.